# SOLID Principles in C#

A practical overview with code examples for object-oriented design

---

SOLID is an acronym for five design principles that help developers write maintainable, understandable, and flexible object-oriented code. Introduced by Robert C. Martin, these principles guide how classes and modules should be structured and how they should interact.

| Letter | Principle | Core Idea |
|---|---|---|
| S | Single Responsibility | A class should have only one reason to change |
| O | Open/Closed | Open for extension, closed for modification |
| L | Liskov Substitution | Subtypes must be substitutable for base types |
| I | Interface Segregation | Prefer many small interfaces over one large one |
| D | Dependency Inversion | Depend on abstractions, not concretions |

# S — Single Responsibility Principle (SRP)

A class should have only one reason to change, meaning it should have only one job or responsibility. Mixing concerns like business logic, persistence, and formatting in one class makes it fragile and hard to maintain.

### Before: violates SRP

```
public class Invoice
{
    public decimal CalculateTotal() { /* ... */ return 0m; }

    public void SaveToDatabase()
    {
        // Persistence logic mixed into the Invoice class
    }

    public void Print()
    {
        // Formatting/printing logic mixed in too
    }
}
```

### After: each class has one responsibility

```
public class Invoice
{
    public decimal CalculateTotal() { /* ... */ return 0m; }
}

public class InvoiceRepository
{
    public void Save(Invoice invoice) { /* persistence logic */ }
}

public class InvoicePrinter
{
    public void Print(Invoice invoice) { /* formatting logic */ }
}
```

# O — Open/Closed Principle (OCP)

Software entities should be open for extension but closed for modification. New behavior should be added by writing new code (e.g. new classes), not by editing existing, tested code — which reduces the risk of breaking what already works.

### Before: adding a shape means editing this class

```
public class AreaCalculator
{
    public double GetArea(object shape)
    {
        if (shape is Circle c) return Math.PI * c.Radius * c.Radius;
        if (shape is Rectangle r) return r.Width * r.Height;
        // Adding a Triangle means modifying this method again
        throw new ArgumentException("Unknown shape");
    }
```

```
}
```

## After: extend via new classes, no modification needed

```
public abstract class Shape
{
    public abstract double GetArea();
}

public class Circle : Shape
{
    public double Radius { get; set; }
    public override double GetArea() => Math.PI * Radius * Radius;
}

public class Rectangle : Shape
{
    public double Width { get; set; }
    public double Height { get; set; }
    public override double GetArea() => Width * Height;
}

// New shapes are added by creating new classes, e.g. Triangle : Shape
public class AreaCalculator
{
    public double GetArea(Shape shape) => shape.GetArea();
}
```

# L — Liskov Substitution Principle (LSP)

Objects of a superclass should be replaceable with objects of a subclass without breaking the application. Subclasses must honor the behavior expected of their base type, not just its method signatures.

### Before: Square breaks the Rectangle contract

```
public class Rectangle
{
    public virtual int Width { get; set; }
    public virtual int Height { get; set; }
}

public class Square : Rectangle
{
    public override int Width
    {
        set { base.Width = value; base.Height = value; } // Surprising side effect
    }
}
// Code expecting a Rectangle can silently break when given a Square
```

### After: model the actual abstraction correctly

```
public abstract class Shape
{
    public abstract int GetArea();
}

public class Rectangle : Shape
{
    public int Width { get; set; }
    public int Height { get; set; }
    public override int GetArea() => Width * Height;
}

public class Square : Shape
{
    public int Side { get; set; }
    public override int GetArea() => Side * Side;
}
// Square no longer pretends to be a Rectangle it isn't
```

# I — Interface Segregation Principle (ISP)

Clients should not be forced to depend on methods they do not use. Prefer several small, focused interfaces over one large, general-purpose interface.

### Before: one bloated interface

```
public interface IWorker
{
    void Work();
    void Eat();
}

public class RobotWorker : IWorker
```

```
{
    public void Work() { /* ... */ }
    public void Eat() { throw new NotImplementedException(); } // Forced, unused
}
```

### After: split into focused interfaces

```
public interface IWorkable
{
    void Work();
}

public interface IFeedable
{
    void Eat();
}

public class HumanWorker : IWorkable, IFeedable
{
    public void Work() { /* ... */ }
    public void Eat() { /* ... */ }
}

public class RobotWorker : IWorkable
{
    public void Work() { /* ... */ } // No unused Eat() method
}
```

# D — Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules; both should depend on abstractions. Abstractions should not depend on details — details should depend on abstractions. This is what makes dependency injection and mocking possible.

### Before: tightly coupled to a concrete class

```
public class SqlLogger
{
    public void Log(string message) { /* write to SQL */ }
}

public class OrderService
{
    private readonly SqlLogger _logger = new SqlLogger(); // Hard-coded dependency

    public void PlaceOrder()
    {
        _logger.Log("Order placed");
    }
}
```

### After: depend on an abstraction

```
public interface ILogger
{
    void Log(string message);
}

public class SqlLogger : ILogger
{
    public void Log(string message) { /* write to SQL */ }
}

public class ConsoleLogger : ILogger
{
    public void Log(string message) => Console.WriteLine(message);
}

public class OrderService
{
    private readonly ILogger _logger;

    public OrderService(ILogger logger) // Injected, easily swapped or mocked
    {
        _logger = logger;
    }

    public void PlaceOrder()
    {
        _logger.Log("Order placed");
    }
}
```

### Why SOLID matters

Applied together, these principles lead to code that is easier to test, extend, and refactor. They aren't rigid laws — over-applying them (e.g. creating excessive abstractions for simple problems) can add unnecessary complexity, so use judgment based on the size and lifespan of the codebase.